Patrones aplicados al desarrollo de agentes · Cheatsheet

Resumen de los 42 patrones de la guía para arquitectos empresariales y de soluciones

9 de noviembre de 2026 · @whinedo · Hector Gracia de Azcoitia

Licencia: CC BY-SA 4.0. Detalles, contenido de terceros y aviso de responsabilidad en la sección «Licencia y avisos», al final del documento.

Índice

- Resumen
- Patrones
  - Comportamiento: un solo agente (§3)
  - Coordinación: varios agentes (§4)
  - Concurrencia: trabajo a la vez (§5)
  - Sistemas distribuidos: fallos parciales (§6)
  - Seguridad: defensa frente a la prompt injection (§7)
  - Operación: puesta en producción (§8)
- Sobre la autoría
- Licencia y avisos

Resumen

Este cheatsheet condensa en una sola consulta los 42 patrones de la guía «Patrones aplicados al desarrollo de agentes». Para cada patrón recoge su diagrama y la señal que indica cuándo aplicarlo, agrupados en seis familias: comportamiento de un agente, coordinación entre agentes, concurrencia, sistemas distribuidos, seguridad frente a la prompt injection y operación en producción.

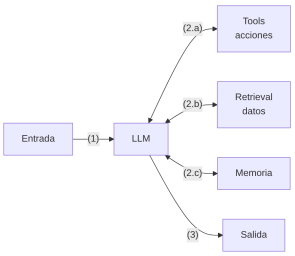
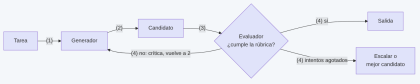
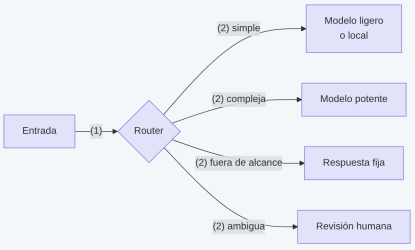
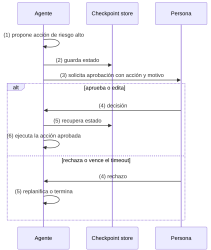
La idea que lo recorre es que un agente es un sistema distribuido con un componente no determinista en el centro. Por eso conviven los patrones clásicos (retry, idempotency key, saga, circuit breaker) con otros propios del LLM. La regla general es empezar por lo más simple (llamada única → workflow → agente → multiagente) y subir solo cuando la eval lo justifique.

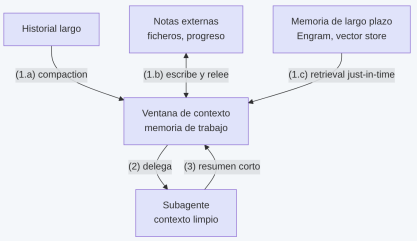
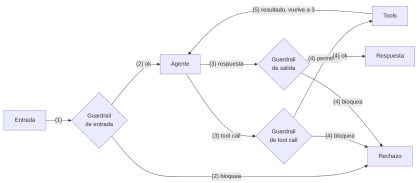
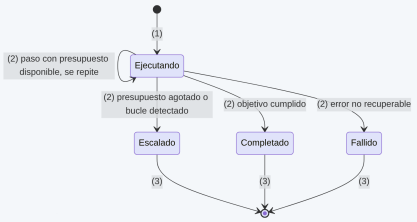
El número de cada fila (§3.1, §4.2…) remite a la sección de la guía con el detalle completo. La columna «Cuándo aplicarlo» sirve como primer filtro, no como sustituto del análisis de compromisos de la guía.

Palabras clave: agentes de IA, patrones de diseño, arquitectura de agentes, multiagente, sistemas distribuidos, prompt injection, evals.

Comportamiento

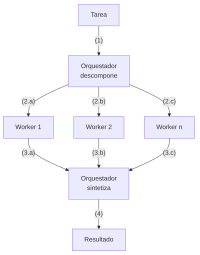
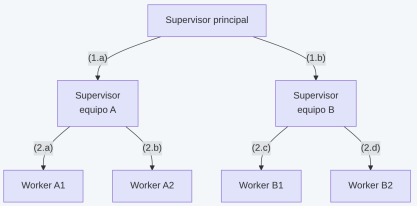
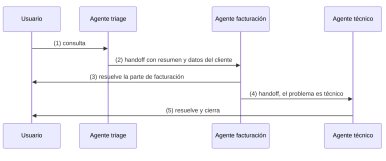
Patrones de un solo agente: cómo razona, actúa y se controla.

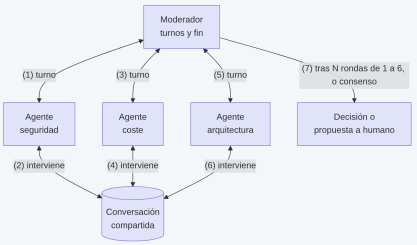
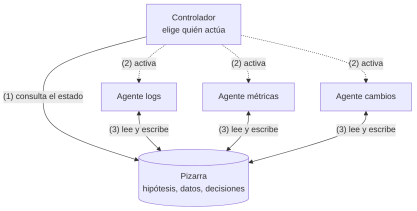
Patrón	Diagrama	Cuándo aplicarlo
Augmented LLM §3.1		Siempre: es el bloque base. Sobra solo si basta el conocimiento del modelo, sin datos ni acciones.
ReAct §3.2		Los pasos dependen de lo que se descubre (búsqueda, diagnóstico). Siempre con control de bucles y presupuesto.
Plan-and-Execute §3.3		Tareas largas con plan visible y aprobable; modelo potente para planificar y barato para ejecutar.
Evaluator-Optimizer §3.4		Hay un criterio de calidad claro (tests, esquema, rúbrica) y la crítica mejora el resultado.
Prompt chaining con gates §3.5		Tarea que se divide en pasos fijos y conocidos, con contratos verificables entre pasos.
Routing §3.6		Entradas de categorías distintas, o mandar lo fácil a un modelo barato o local y lo difícil a uno potente.
Human-in-the-loop §3.7		Acciones irreversibles o caras (pagos, borrados, envíos, producción) o riesgo alto en la spec.

Patrón	Diagrama	Cuándo aplicarlo
Memoria por capas §3.8		Agentes de muchos turnos o documentos grandes: el contexto es un recurso escaso.
Guardrails §3.9		Siempre que el agente reciba entradas de terceros o pueda actuar fuera de su proceso.
Bounded autonomy §3.10		Todo agente con bucle: límites de pasos, tokens, tiempo y coste, con condición de parada.

Coordinación

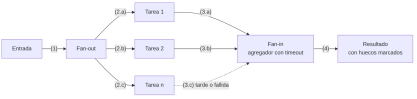
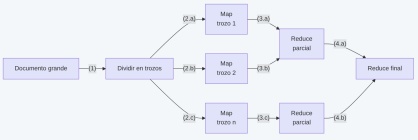
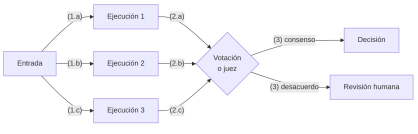
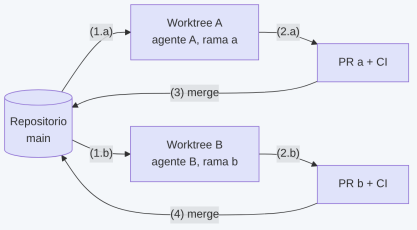
Cómo se reparten el trabajo varios agentes.

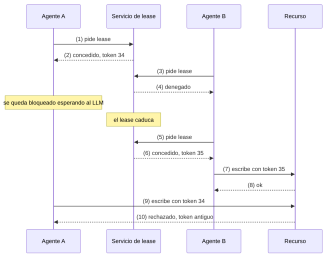
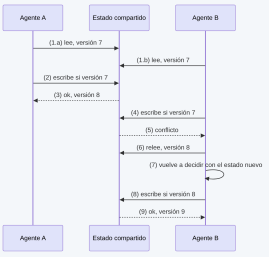
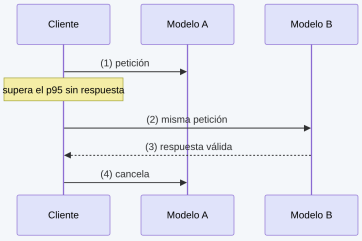
Patrón	Diagrama	Cuándo aplicarlo
Orchestrator-workers §4.1		Subtareas que no se conocen de antemano pero, una vez definidas, son independientes (research).
Jerárquico §4.2		Un supervisor tendría demasiados workers o tools, o los equipos son dominios distintos.
Handoff §4.3		Atención con áreas especializadas que se descubren durante la conversación. Un agente activo cada vez.

Patrón	Diagrama	Cuándo aplicarlo
Group chat / debate §4.4		Revisión desde varios puntos de vista o contraste de hipótesis, con condición de fin explícita.
Blackboard §4.5		Problemas resueltos por aportaciones incrementales de especialistas sin orden fijo (diagnóstico).
Pipeline secuencial §4.6		Proceso con etapas claras y especialistas distintos, con contratos tipados entre etapas.

Concurrencia

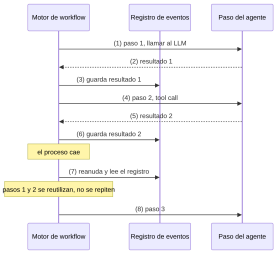
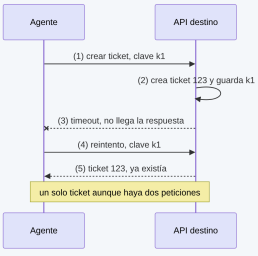
Qué hacer cuando varios agentes o instancias trabajan a la vez.

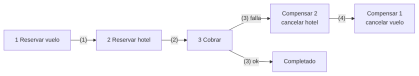
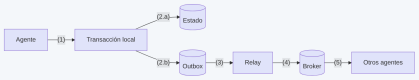
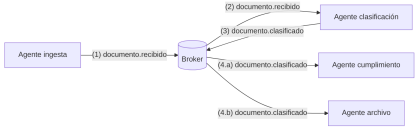
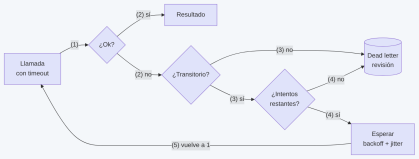
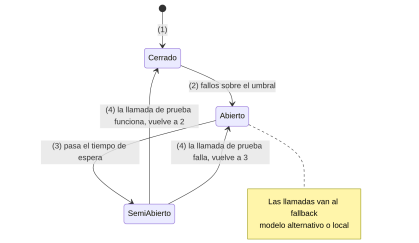
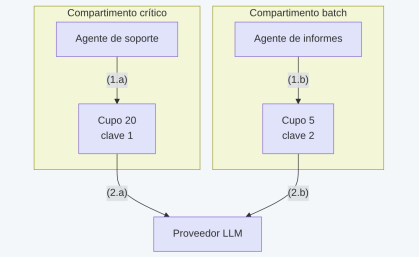
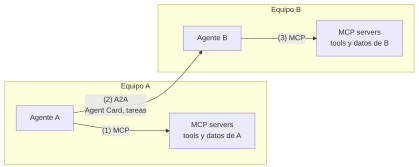
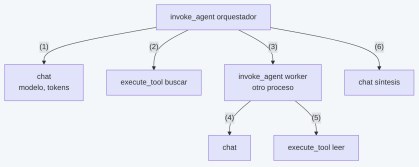
Patrón	Diagrama	Cuándo aplicarlo
Fan-out / fan-in §5.1		Subtareas independientes donde importa la latencia: varias fuentes, ficheros o controles.
Map-reduce §5.2		Corpus que no cabe en el contexto, sobre todo con modelos locales de contexto limitado.
Voting §5.3		Decisiones discretas de alto impacto donde un error cuesta más que varias llamadas.
Aislamiento del workspace §5.4		Varios agentes modifican el mismo repositorio o recurso a la vez.

Patrón	Diagrama	Cuándo aplicarlo
Lease con fencing token §5.5		Solo un agente debe actuar sobre un recurso y los agentes pueden colgarse o tardar.
Control optimista §5.6		Estado compartido con conflictos poco frecuentes (pizarra, documento de tareas).
Backpressure §5.7		Fan-out o colas contra un LLM con rate limits, o un modelo local con pocas peticiones simultáneas.
Ejecución especulativa §5.8		La latencia de cola importa más que el coste, solo con operaciones idempotentes.

Sistemas distribuidos

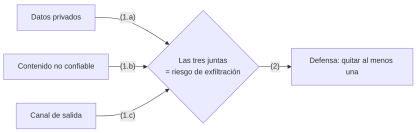
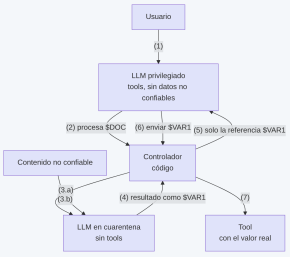
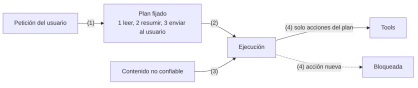
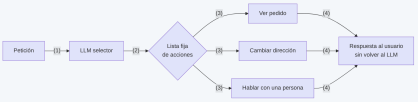
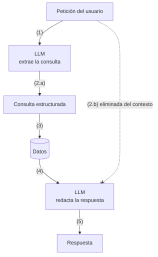
Cómo sobrevivir a fallos parciales, red y no determinismo.

Patrón	Diagrama	Cuándo aplicarlo
Durable execution §6.1		Agentes de larga duración, con aprobaciones humanas o pasos caros o con efectos.
Idempotency key §6.2		Toda tool que crea, modifica o envía algo (pagos, tickets, correos, despliegues).

Patrón	Diagrama	Cuándo aplicarlo
Saga §6.3		Acciones con efectos en varios sistemas sin transacción común.
Transactional outbox §6.4		El agente actualiza su estado y debe avisar a otros sin que ocurra lo uno sin lo otro.
Coreografía por eventos §6.5		Agentes de equipos distintos y muy desacoplados, procesos que crecen añadiendo consumidores.
Retry con backoff y dead letter §6.6		Todas las llamadas remotas (LLM, MCP, APIs). Nunca con errores permanentes.
Circuit breaker con fallback §6.7		Dependencias externas que se degradan y existe una alternativa evaluada (otro modelo, local).
Bulkhead §6.8		Conviven agentes críticos e interactivos con batch, o varios equipos comparten proveedor.
MCP y A2A §6.9		MCP: tools reutilizables entre agentes. A2A: agentes de equipos distintos o de terceros.
Tracing distribuido §6.10		Desde el primer agente que sale del portátil, y siempre en multiagente.

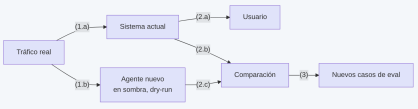
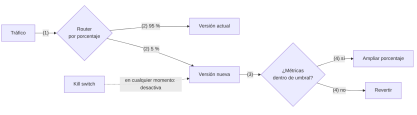
Seguridad

Defensas de arquitectura contra la prompt injection.

Patrón	Diagrama	Cuándo aplicarlo
Lethal trifecta §7.1		Revisión de diseño de cualquier agente, antes de elegir sus tools.
Dual LLM §7.2		El agente procesa contenido no confiable y además ejecuta acciones.
Plan-then-execute §7.3		El plan se deduce de la petición del usuario, antes de leer contenido externo.
Action-selector §7.4		Traducir lenguaje natural a una acción conocida, sin encadenar pasos.
Context minimization §7.5		Chatbots sobre datos: la petición se convierte en consulta y se descarta el texto original.

Operación

Cómo se pone en producción con confianza.

Patrón	Diagrama	Cuándo aplicarlo
Evals como quality gates §8.1		Siempre: cualquier cambio de prompt, modelo, tool o patrón.
Shadow mode §8.2		Antes de sustituir un proceso existente o cambiar de modelo en un agente crítico.
Canary con kill switch §8.3		Cualquier cambio de modelo, prompt o tool en un agente en producción.

Sobre la autoría

Autor: Hector Gracia de Azcoitia (@whinedo).

Licencia y avisos

Licencia

© 2026 Hector Gracia de Azcoitia. Este documento se distribuye bajo la licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional (CC BY-SA 4.0). Puedes copiarlo, redistribuirlo y adaptarlo para cualquier fin, también comercial, con dos condiciones:

- **Atribución:** reconocer la autoría, enlazar la licencia e indicar si has hecho cambios.
- **Compartir igual:** distribuir tus adaptaciones con esta misma licencia.

Atribución sugerida: «Patrones aplicados al desarrollo de agentes · Cheatsheet», de Hector Gracia de Azcoitia, licencia CC BY-SA 4.0.

Contenido de terceros

- Los patrones se describen con palabras propias a partir de la guía «Patrones aplicados al desarrollo de agentes» y de las fuentes que esta cita; esas fuentes conservan sus propios derechos y no quedan cubiertas por esta licencia.
- Los nombres de patrones, protocolos (MCP, A2A) y marcos se mencionan por su nombre; sus especificaciones pertenecen a sus editores.
- Las marcas y nombres de productos pertenecen a sus titulares. Su mención no implica afiliación ni respaldo.

Aviso de responsabilidad

Este cheatsheet tiene fines formativos y divulgativos. Su contenido es orientativo, puede contener errores y no sustituye el detalle de la guía ni una evaluación profesional de la arquitectura. Las herramientas, protocolos y prácticas citados cambian con rapidez, así que conviene verificar su estado antes de aplicar cualquier recomendación. El autor no se responsabiliza de las decisiones tomadas a partir de este documento.